



Animations

Overview

The chapter covers three separate tools that all make a game feel alive: **frame animations** from sprite sheets (`this.anims` , `play` , animation events), **tweens** (`this.tweens` : `easing`, `yoyo`, `repeat`, `chains`, `stagger`, `counters`) and **particles** (`this.add.particles` , `explode`). The design idea is the **state machine** for a character's animations - the first time in the book that an object's behaviour is organised around explicit states, and a pattern students will meet again in Chapters 12, 13, 15 and 17. The TypeScript ideas are **string-literal union types** as enums, `Record<K, V>` , `switch` on strings, `Partial<T>` , `??` and a justified `as` .

It is a full chapter. If time is short, the lab and the hero are the core; the tween playground can be explored by students on their own with the README beside them.

Prerequisites

- Chapter 1: `preUpdate` and `delta` , speeds in pixels per second, origins
- Chapter 2: `on / once` , `setInteractive` , scenes reused and reset in `init()`
- Chapter 3: loading sprite sheets (a quick recap is in the chapter)
- Chapter 4: `Sprite.preUpdate` and why `super.preUpdate` matters (the chapter points back to it)
- Chapter 6: floating text with a tween is referred to in passing

Learning outcomes

Students can:

1. make named animations from a sprite sheet with `this.anims.create` and `generateFrameNumbers` , choosing `frameRate` , `repeat` and `yoyo` , and explain why they are made once, in a preload scene

2. play and control an animation on a sprite (`play` , `play(key, true)` , `chain` , `playAfterRepeat` , `stop` , `pause` , `timeScale` , `setFlipX`) and read its state (`currentAnim` , `currentFrame`)
3. react to animation events, including `ANIMATION_COMPLETE_KEY + key`
4. structure a character's animations as a state machine with a single `changeState` method
5. write tweens with `duration` , `ease` , `yoyo` , `hold` , `repeat` , `delay` and `onComplete` ; use chains, stagger and counters; and prevent tweens fighting with `killTweensOf`
6. choose a suitable ease, and describe the difference between `easeIn`, `easeOut` and `easeInOut`
7. make a burst of particles with an emitter created once with `emitting: false`

Suggested session plan (1 hour lecture + 2 hour lab)

Lecture (1 hour)

Time	Activity
0:00 - 0:05	Show a still of the hero next to the running game. "What is different?" - frames, flipping, and movement
0:05 - 0:20	Slides 3-6: frame animation vs tween; the sprite sheet diagram; <code>animations.create</code> ; global animations; Sprite vs Image
0:20 - 0:30	Live demo in <code>ch07_animation_lab</code> : press 2 repeatedly, then hold it - the hero freezes on frame 2. Then 3. Ask for an explanation before showing slide 7
0:30 - 0:40	Slides 8-9: chain, playAfterRepeat, events. In the lab: C, then 2 followed by A, watching the event log
0:40 - 0:55	Slides 10-13: the state machine. Draw the states on the board with the class first, then show <code>changeState</code> and <code>updateState</code>
0:55 - 1:00	Slides 14 and 18 as a preview of tweens and particles, for the lab

Lab (2 hours)

Time	Activity
0:00 - 0:15	Build and play all three projects. In the hero project, read <code>Hero.ts</code> from top to bottom in pairs
0:15 - 0:30	Challenge 1 (frame rates) - everyone
0:30 - 0:50	Tweens: slides 14-18 on screen; students press every button in the playground, then read the matching method. The easing gallery: 1, 2, 3
0:50 - 1:00	Live demo: comment out <code>resetCrate()</code> in <code>pulse()</code> , press Pulse four times quickly - the crate ends up stuck at a bigger size (2.4 instead of 1.5). Ask why
1:00 - 1:45	Challenges 2-4, then 5 and 6 for those who are ahead
1:45 - 2:00	Show-and-tell of title sequences (challenge 3) and stomping (challenge 6)

Key points to stress

- **frames vs tweens.** A frame animation changes the picture; a tween changes a number. Students often try to "tween the frames" or "animate the position" - naming the difference early avoids it
- **animations belong to the game.** `this.anims` is the game's animation manager in every scene; making animations in a scene that restarts gives a console warning, not an error, so it can go unnoticed for a long time
- **play restarts.** The single most common animation bug is `play(key)` in `update()`, which restarts the animation every frame so it never gets past its first frame. Two cures: `play(key, true)`, or (better) only call `play` when the state changes
- **one place changes state.** The state machine's value comes from `changeState` being the only place the state and the animation change. If students set `currentState` in several places, the benefit is lost
- `super.preUpdate`: a `Sprite` subclass that overrides `preUpdate` without calling `super` shows a frozen frame and gives no error at all
- **ask, don't reach in** (from Chapter 2) returns in the hero: the scene calls `hero.hurt()`; it does not set the hero's velocity or state

- **effects clean up after themselves:** `onComplete: () => popup.destroy()`, and `once(ANIMATION_COMPLETE, () => boom.destroy())`. Otherwise the scene fills with invisible objects
- **make an emitter once**, and use `explode` or `emitting` - never a new emitter per burst

Common problems and errors

What students see	Cause	Fix
The character slides about frozen on one frame; no error	<code>preUpdate</code> overridden without <code>super.preUpdate(time, delta)</code>	call <code>super.preUpdate</code> first
The animation is stuck on its first frame while a key is held	<code>play(key)</code> called every frame (or on every key repeat) restarts it	<code>play(key, true)</code> , or play only when the state changes
Property 'play' does not exist on type 'Image'.	the object was made with <code>this.add.image</code>	<code>this.add.sprite</code> (or extend <code>Sprite</code>)
The sprite keeps its old animation; console: Missing animation: hero-run	misspelled animation key	keep animation keys as constants
Console: AnimationManager key already exists: hero-idle	animations made in a scene that starts more than once	make them in the preload scene
This member must have an 'override' modifier because it overrides a member in the base class 'Sprite'.	<code>preUpdate</code> in a <code>Sprite</code> subclass without <code>override</code>	add <code>override</code> (and keep it <code>protected</code>)
Class 'Hero' incorrectly extends base class 'Sprite'. Property 'state' is private in type 'Hero' but not in type 'Sprite'.	a field named <code>state</code> - every game object already has one	another name, e.g. <code>currentState</code> (likewise a method called <code>setState</code>)
Property 'hurt' is missing in type '{ idle: string; run: string; jump: string; fall: string; }' but required in type 'Record<HeroState, string>'.	a state added to the union but not to the <code>Record</code>	this is the type doing its job - add the animation
An <code>ANIMATION_COMPLETE</code> listener never runs	the animation has <code>repeat: -1</code> and never completes	give it a finite <code>repeat</code> , or listen for <code>ANIMATION_REPEAT</code> / <code>ANIMATION_STOP</code>

What students see	Cause	Fix
After several hurts, the "hurt is over" code runs several times at once	<code>on</code> used where <code>once</code> was meant, adding a listener per hurt	<code>once</code>
A yoyo tween "comes back" to the wrong place, or the object creeps	a second tween started while the first was running, from a mid-way value	<code>killTweensOf</code> and reset first, or ignore the input while <code>isTweening</code>
The ease looks linear	misspelled ease string (<code>"Bounce.easout"</code>) - Phaser silently falls back to linear	check the spelling; no error is given
Argument of type 'number null' is not assignable to parameter of type 'number'. on <code>Math.round(tween.getValue())</code>	<code>getValue()</code> returns <code>number null</code>	<code>tween.getValue()</code> <code>?? 0</code>
Particles appear at (0, 0)	<code>explode(count)</code> with no position, on an emitter made at (0, 0)	<code>explode(count, x, y)</code>
The game slows down after a minute of the trail/explosions	a new emitter made for every burst or every pointer move	make one emitter, in <code>create()</code>

The error messages above were produced in scratch copies of the chapter projects.

Suggested demos and live-coding moments

- **The stuck run.** In the lab, hold 2 then hold 3. Nothing explains `ignoreIfPlaying` as well
- **Delete** `super.preUpdate` in `Hero.ts`, build, run right. The hero glides, frozen. Put it back
- **Change one number:** the hurt animation's `frameRate` from 8 to 3. The hero stays hurt for longer, with no other change - the animation event ties game rules to animations
- **Pulse without** `resetCrate` (see the lab plan), then with it

- **Easing on the board.** Before showing the gallery, ask students to sketch how a door, a ball dropped on the floor, and a menu sliding in should move. Then find their curves in the gallery
 - **Particle knobs:** in `addExplosions`, change `gravityY` to -300, `speed` to a single number, remove `blendMode: "ADD"`, make `quantity` 200. Discuss what each setting does to the "feel"
-

Discussion questions

- The hero has five states. Which pairs of states can never follow each other directly? Where does the code make sure of that?
 - Why is it `this.once(...)` in `hurt()` and not `this.on(...)` in the constructor? Could it be `on` in the constructor instead? (Yes - with the listener registered once. Both are defensible; discuss which is easier to follow.)
 - What is the difference between `animations.timeScale` on a sprite and changing `frameRate` in `animations.create`?
 - When would you use a tween to move a character, and when physics or `preUpdate`? (Tweens: a known start, end and time - cut-scenes, menus, platforms on rails. Physics: anything that responds to the player or to collisions.)
 - `killTweensOf` then reset, or ignore the click while `isTweening`? Which feels better for a button? For a character's attack?
-

Extension ideas

- give the hero a **land** state with a short squash (a `scaleY` tween, yoyo) between fall and idle
- add `this.animations.addMix(HERO_RUN, HERO_IDLE, 100)` and compare stopping with and without it
- a **particle trail** from the hero's feet while running (a flowing emitter with `startFollow`, switched by the state machine)
- use `animations.staggerPlay` to start a row of coins spinning out of step
- tween along a path: `Phaser.Curves.Path` and `this.add.follower` for an enemy that flies loops

- explore `Stepped` easing (`ease: "Stepped", easeParams: [5]`) for a deliberately jerky, retro look
-

Assessment ideas

- **Practical:** "add a `duck` state to the hero: DOWN held on the ground shows frame 10 and stops running; releasing DOWN returns to idle". Marks for: the union, the `Record` , the transitions in `updateState` , no `play` outside `changeState`
 - **Code reading:** show a `Sprite` subclass with `play(RUN)` in `preUpdate` and no `super` call; ask for both bugs and their symptoms
 - **Short answer:** explain `yoyo` , `hold` , `repeat: 2` and `repeatDelay` by sketching the value against time (the timeline diagram in the chapter is the model answer)
 - **Design:** draw the state machine for a door (closed, opening, open, closing, locked), naming the animation and the event that ends each state
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. Different speeds

Project: `solutions/ch07_challenge_1_different_speeds` (from `ch07_animation_lab`)

Goal: find where an animation's speed is decided, and see that a new animation needs no new art.

`src/scenes/PreloadScene.ts`

```
this.anns.create({
  key: BAT_FLAP,
  frames: this.anns.generateFrameNumbers(BAT_SHEET, { start: 0, end: 3 }),
  frameRate: 24, // CHALLENGE 1: twice as fast (was 12)
  repeat: -1,
});
...
// CHALLENGE 1: the same frames as hero-run, faster - a new animation needs
// no new pictures
this.anns.create({
  key: HERO_SPRINT,
  frames: this.anns.generateFrameNumbers(HERO_SHEET, { start: 2, end: 7 }),
  frameRate: 20,
  repeat: -1,
});
```

`src/scenes/LabScene.ts`

```
keyboard.on("keydown-SEVEN", () => this.hero.play(HERO_SPRINT)); //
CHALLENGE 1
```

plus the ghost's `frameRate: 2`, the new `HERO_SPRINT` key in `assets.ts`, the two labels, the help line and `index.html`.

Look for: the new key added to `assets.ts` as a constant, not typed as a string in two places. Changing the frame rate *where the animation is made* rather than setting `timeScale` on the sprite - both work, but only one changes the animation for every sprite that plays it. Tested with the harness: key 7 plays `hero-sprint` at 20 fps; `game.anims.get("bat-flap").frameRate` is 24.

2. Coin run

Project: `solutions/ch07_challenge_2_coin_run` (from `ch07_hero_animations`)

Goal: load a second sprite sheet and animation; a simple distance test; a tween that cleans up after itself.

`coin_spin.png` is copied into `public/assets/spritesheets/`, loaded in `PreloadScene`, and given a looping `coin-spin` animation there. The scene makes the coins:

`src/scenes/GameScene.ts`

```
// CHALLENGE 2: a row of spinning coins, alternately low and high
COIN_XS.forEach((x, i) => {
  const y = i % 2 === 0 ? COIN_LOW_Y : COIN_HIGH_Y;
  const coin = this.add.sprite(x, y, COIN_SHEET).setScale(COIN_SCALE);
  coin.play(COIN_SPIN);
  this.coins.push(coin);
});
```

and checks them every frame, measuring from the middle of the hero's body (its `y` is its feet):

```

private checkCoins(): void {
    const heroMiddleY = this.hero.y - this.hero.displayHeight / 2;
    for (const coin of [...this.coins]) {          // a copy: collect()
        removes coins from the list
        const distance = Phaser.Math.Distance.Between(this.hero.x, heroMiddleY,
        coin.x, coin.y);
        if (distance < PICKUP_DISTANCE) {
            this.collect(coin);
        }
    }
}

private collect(coin: Phaser.GameObjects.Sprite): void {
    this.coins = this.coins.filter((c) => c !== coin);
    this.collected = this.collected + 1;
    this.coinText.setText(`Coins: ${this.collected} / ${COIN_XS.length}`);

    this.tweens.add({
        targets: coin,
        y: coin.y - 80,
        alpha: 0,
        scale: COIN_SCALE * 1.5,
        duration: 500,
        ease: "Quad.easeOut",
        onComplete: () => {
            coin.destroy();
        },
    });
}

```

Look for:

- the coin taken **out of the list at once**, before its tween. The most common bug is to destroy it only in `onComplete` while it is still in the list: the next frames count it again, and one coin scores ten or twenty times
- `destroy()` in `onComplete` (or the faded coins stay in the scene)
- `init()` resetting the list and the count, in case the scene is restarted (Chapter 2's trap)
- measuring from the hero's `y` (its feet) without noticing - it half works, which makes it hard to spot. A good moment to revisit origins from Chapter 1

Tested with the harness: running along the ground collects the low coins; jumping collects the high ones; the count matches.

3. Title sequence

Project: `solutions/ch07_challenge_3_title_sequence` (from `ch07_tweens_and_particles`)

Goal: a `tweens.chain` of several steps, and making it safe to restart.

`src/scenes/PlaygroundScene.ts`

```
private playTitle(): void {
  this.tweens.killTweensOf(this.title);
  this.title.setPosition(TITLE_X,
    TITLE_START_Y).setScale(1).setAngle(0).setAlpha(1);

  this.tweens.chain({
    targets: this.title,
    tweens: [
      { y: TITLE_Y, duration: 1000, ease: "Bounce.easeOut" },
      { scale: 1.3, duration: 200, ease: "Quad.easeOut", yoyo: true },
      { angle: { from: -6, to: 6 }, duration: 100, yoyo: true, repeat: 3 },
      { angle: 0, duration: 60 },
      { alpha: 0, delay: 800, duration: 500 },          // delay: hold still,
    ],
    then fade
  });
  this.show("T: a chain of five tweens on the title");
}
```

The title is made once in `create()`, above the top of the screen, and T calls `playTitle()`.

- `{ from, to }` sets the start value as well as the end: the wobble starts at -6 degrees wherever the angle was
- `delay` on the last tween is the "hold for a moment"; a `hold` on the tween before would also work
- `killTweensOf` stops the chain as well as single tweens

Look for: the reset before the chain - without it, a second T part way through starts from wherever the title had got to (half faded, or tilted). Accept `onComplete` callbacks nested

inside each other, but show the chain as the tidier answer. Tested with the harness: T, then T again after half a second, leaves exactly one tween running and the title lands at `y = 110`.

4. Sparkle trail

Project: `solutions/ch07_challenge_4_sparkle_trail` (from `ch07_tweens_and_particles`)

Goal: a flowing emitter, following something, turned on and off.

`src/scenes/PlaygroundScene.ts`

```
private addTrail(): void {
  this.trail = this.add.particles(0, 0, PARTICLE_KEY, {
    speed: { min: 5, max: 40 },
    lifespan: 600,
    scale: { start: 1.5, end: 0 },
    alpha: { start: 0.8, end: 0 },
    tint: [0xa8dadc, 0xffffffff, 0xffd166],
    blendMode: "ADD",
    frequency: 15,           // a burst every 15 milliseconds...
    quantity: 2,             // ...of two particles
    emitting: false,         // not until the pointer is in the playground
  });
  this.trail.startFollow(this.input.activePointer);

  // only in the playground: stop over the buttons (and when the pointer
  // leaves the game)
  this.input.on("pointermove", (pointer: Phaser.Input.Pointer) => {
    this.trail.emitting = pointer.x > PLAY_LEFT;
  });
  this.input.on("gameout", () => {
    this.trail.emitting = false;
  });
}
```

- the explosion emitter uses `emitting: false` + `explode()`; this one **flows** (`frequency`, `quantity`) and is switched with `emitting`

- `startFollow` takes anything with an `x` and a `y` - the pointer, a sprite, a plain object
- particles already alive carry on when `emitting` goes false, so the trail fades out rather than vanishing

Look for: one emitter, made once. A common wrong answer calls

`this.add.particles(...)` inside `pointermove`, making a new emitter dozens of times a second - it works for a minute, then the game slows to a crawl. Another is

`emitter.stop() / start()` every move; it works, but `emitting` says what is meant.

Tested with the harness: moving across the playground gives 60+ live particles; moving onto the buttons sets `emitting` to false and they are all gone within a second.

5. Speed you can see

Project: `solutions/ch07_challenge_5_speed_you_can_see` (from `ch07_hero_animations`)

Goal: acceleration with `delta`, and an animation's playback speed driven by the game.

`src/objects/Hero.ts`

```

private readKeys(seconds: number): void {
    let direction = 0;
    if (this.cursors.left.isDown) {
        direction = -1;
    } else if (this.cursors.right.isDown) {
        direction = 1;
    }

    if (direction !== 0) {
        // pushing against the way the hero is going: brake AND accelerate, for
        a quick turn
        const turning = this.velocityX * direction < 0;
        const rate = turning ? ACCELERATION + DECELERATION : ACCELERATION;
        this.velocityX = Phaser.Math.Clamp(this.velocityX + direction * rate *
seconds, -RUN_SPEED, RUN_SPEED);
    } else {
        // slow down towards 0 - and stop exactly AT 0, not just past it
        const slowBy = DECELERATION * seconds;
        if (Math.abs(this.velocityX) <= slowBy) {
            this.velocityX = 0;
        } else {
            this.velocityX = this.velocityX - Math.sign(this.velocityX) * slowBy;
        }
    }
    ...
}

```

```

private matchStrideToSpeed(): void {
    if (this.currentState === "run") {
        this.anims.timeScale = Math.max(MIN_TIME_SCALE, Math.abs(this.velocityX)
/ RUN_SPEED);
    } else {
        this.anims.timeScale = 1;
    }
}

```

`matchStrideToSpeed()` is called at the end of `preUpdate()`. The state machine did not change: the hero is "run" while `velocityX` is not 0, and "idle" when it is.

Look for:

- slowing down that **snaps to exactly 0**. Without it the speed wobbles either side of zero for ever (-0.3, 0.2, -0.1...), the hero never becomes idle, and it twitches left and right as

the flip follows the sign

- `timeScale` put back to 1 in other states - otherwise the next animation keeps the last run speed (a jump after a slow start plays slowly)
- a minimum `timeScale` : at a speed of 2 pixels a second, a `timeScale` of 0.008 looks frozen
- `delta` used for both acceleration and deceleration

Tested with the harness: holding RIGHT, the speed goes 50, 140, 260 with `timeScale` 0.30, 0.54, 1.00; after letting go the hero is still `run` at 170, then `idle` at 0.

6. Stomp

Project: `solutions/ch07_challenge_6_stomp` (from `ch07_hero_animations`)

Goal: bring the chapter together - a second animated class, animation events, particles - and keep the state machine in charge of the hero.

A new `Slime` class (`src/objects/Slime.ts`) is a `Sprite` that patrols between two x positions in its own `preUpdate()` - calling `super.preUpdate()`. `PreloadScene` loads `slime.png`, `explosion.png` and `particle.png` and makes `slime-walk` and `explode`. `Hero` gains three things the scene can ask for:

`src/objects/Hero.ts`

```
// CHALLENGE 6: true while coming down from a jump - the only time a slime
// can be stomped
public isFalling(): boolean {
    return this.currentState === "fall";
}

// CHALLENGE 6: called by the scene after a stomp - spring back up off the
// slime
public bounceOff(): void {
    this.velocityY = -STOMP_BOUNCE;
    this.changeState("jump");
}
```

and `hurt(fromX?: number)`, which knocks the hero away from where the hurt came from. The scene decides what a touch means:

`src/scenes/GameScene.ts`

```
private checkSlimes(): void {
  const heroBounds = this.hero.getBounds();
  for (const slime of [...this.slimes]) {           // a copy: squash()
    removes slimes from the list
    if (!Phaser.Geom.Intersects.RectangleToRectangle(heroBounds,
    slime.getBounds())) {
      continue;
    }
    if (this.hero.isFalling()) {
      this.squash(slime);
      this.hero.bounceOff();
    } else {
      this.hero.hurt(slime.x);                       // does nothing if
      already hurt
    }
  }
}

private squash(slime: Slime): void {
  this.slimes = this.slimes.filter((s) => s !== slime);
  const centreY = slime.y - slime.displayHeight / 2;

  const boom = this.add.sprite(slime.x, centreY, EXPLOSION_SHEET);
  boom.play(EXPLODE);
  boom.once(Phaser.Animations.Events.ANIMATION_COMPLETE, () => {
    boom.destroy();
  });
  this.sparks.explode(SPARKS, slime.x, centreY);

  slime.destroy();
  this.showSlimesLeft();
}
```

- "cannot be hurt again until the hurt state is over" comes free: `hurt()` already returns at once if the hero is hurt
- `isFalling()` is "fall", not "`velocityY > 0`": a hero knocked into the air by a hurt is falling too, but should not stomp

- `bounceOff()` goes through `changeState`, so the jump animation plays and the state machine stays the only thing that changes the hero's state

Look for:

- the scene **asking** the hero (`isFalling()`) and **telling** it (`bounceOff()` , `hurt()`), not setting `hero.velocityY` - the fields are private, so a student who tries gets `Property 'velocityY' is private and only accessible within class 'Hero'`. Some will make the fields public to get round it; this is the moment to talk about why not
- a separate `Slime` class, rather than slime movement written into the scene's `update()`
- the explosion sprite destroying itself, and one emitter reused for every squash
- the slime list copied before looping (`[...this.slimes]`), because squashing removes from it

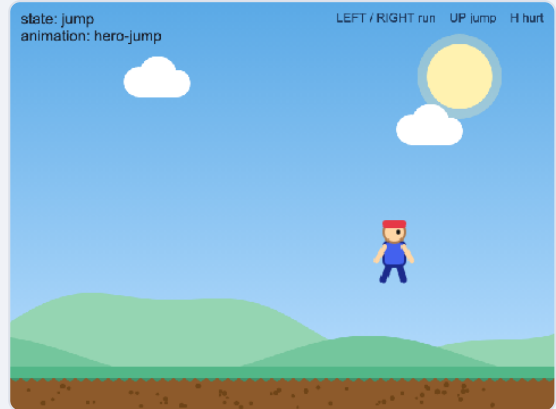
Tested with the harness: dropping the hero onto a slime removes it, bounces the hero into `jump`, and shows "Slimes left: 2"; walking into a slime gives `hurt` with the knockback away from it. With the keys, running left and jumping after 0.1 seconds lands on the first slime.

Slides

Chapter 7

Animations

Frames, tweens and particles



1 / 20

Today

- sprite sheets become **animations**: `this.ans.create`
- playing, chaining and listening to animations
- `setFlipX` , and a **state machine** for the hero
- **tweens**: smooth changes to any number
- easing, chains, stagger, counters
- **particles**: bursts of sparks

2 / 20

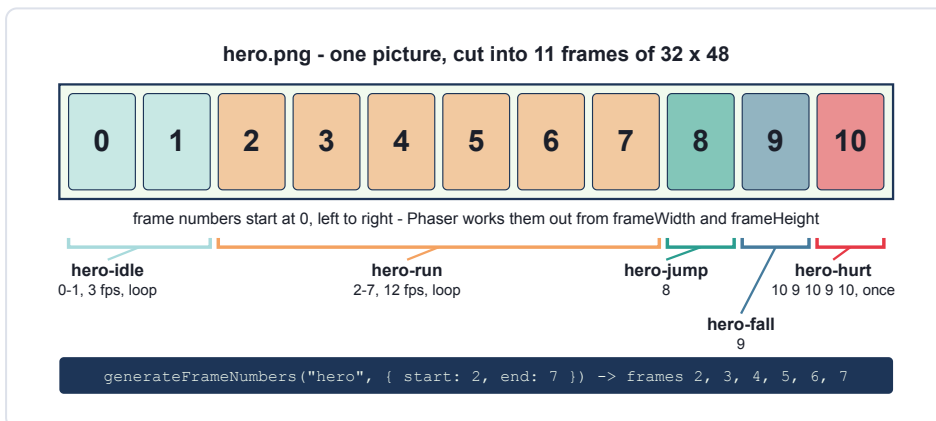
Two kinds of animation

Frame animation	Tween
changes which picture a sprite shows	changes a number smoothly
frames from a sprite sheet	<code>x</code> , <code>y</code> , <code>scale</code> , <code>alpha</code> , <code>angle</code> , ...
a running hero, a spinning coin	a sliding menu, a fading popup
<code>this.ans</code> , <code>sprite.play(key)</code>	<code>this.tweens.add({...})</code>

A character usually needs both.

3 / 20

Sprite sheets



```
this.load.spritesheet(HERO_SHEET, HERO_FILE, { frameWidth: HERO_WIDTH, frameHeight: HERO_HEIGHT });
```

4 / 20

Making an animation

```
this.anims.create({
  key: HERO_RUN, // the animation's own name
  frames: this.anims.generateFrameNumbers(HERO_SHEET, { start: 2, end: 7 }),
  frameRate: 12, // frames per second
  repeat: -1, // -1: for ever
});

this.anims.create({
  key: HERO_HURT,
  frames: this.anims.generateFrameNumbers(HERO_SHEET, { frames: [10, 9, 10, 9, 10] }),
  frameRate: 8, // no repeat: plays once, then COMPLETES
});
```

Also: `yoyo: true`, `repeatDelay`, `delay`, `duration`

5 / 20

Once, for the whole game - on a Sprite

- `this.anims` is the **game's** animation manager, in every scene
- make animations **once**, in the preload scene
- twice? `AnimationManager` key already exists: `hero-idle`

```
const sprite = this.add.sprite(x, 72, sheet); // a Sprite can play
sprite.play(anim);

this.add.image(x, 505, HERO_SHEET, frame) // an Image shows ONE frame
```

`image.play(...)` - Property 'play' does not exist on type 'Image'

6 / 20

play restarts

```
// play() starts the animation again from its first frame - even if it is already playing
keyboard.on("keydown-TWO", () => this.hero.play(HERO_RUN));
// play(key, true): "ignore this if that animation is already playing"
keyboard.on("keydown-THREE", () => this.hero.play(HERO_RUN, true));
```

- `play(key)` in `update()` = stuck on the first frame for ever
- lab: hold **2**, then hold **3**

7 / 20

Controlling, and listening

```
this.hero.play(HERO_HURT);
this.hero.chain(HERO_IDLE); // then idle, automatically

keyboard.on("keydown-A", () => this.hero.ans.playAfterRepeat(HERO_IDLE));
this.hero.ans.timeScale = Math.min(this.hero.ans.timeScale * 2, MAX_TIME_SCALE);

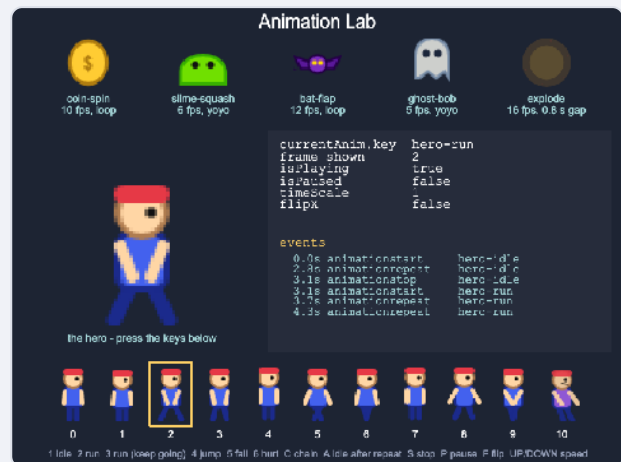
this.hero.on(Phaser.Animations.Events.ANIMATION_COMPLETE_KEY + HERO_HURT, () => {
  this.addToLog(" (hurt is over)");
});
```

repeat: -1 never completes - no `ANIMATION_COMPLETE`

8 / 20

Try it: the Animation Lab

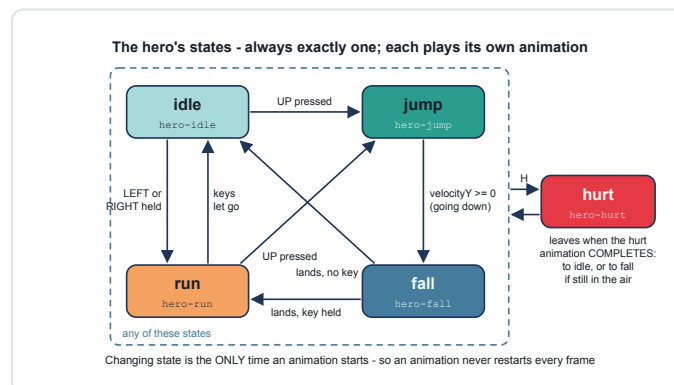
- keys **1-6**: play each animation
- **C** chain, **A** after repeat, **S** stop, **P** pause
- **F** flip, **UP/DOWN** speed
- watch the **event log**



9 / 20

Facing both ways - and a state machine

```
if (this.velocityX !== 0) {
  this.setFlipX(this.velocityX < 0); // frames face right; mirrored = left
}
```



10 / 20

States in code

```
export type HeroState = "idle" | "run" | "jump" | "fall" | "hurt";

const ANIMATIONS: Record<HeroState, string> = {
  idle: HERO_IDLE,
  run: HERO_RUN,
  ...
};

private changeState(next: HeroState): void {
  if (next === this.currentState) {
    return; // already in that state: do NOT restart its animation
  }
  this.currentState = next;
  this.play(ANIMATIONS[next]);
}
```

The **only** place the state - and the animation - changes

11 / 20

Hurt: ended by the animation

```
public hurt(): void {
  ...
  this.changeState("hurt");
  this.setTint(HURT_TINT);

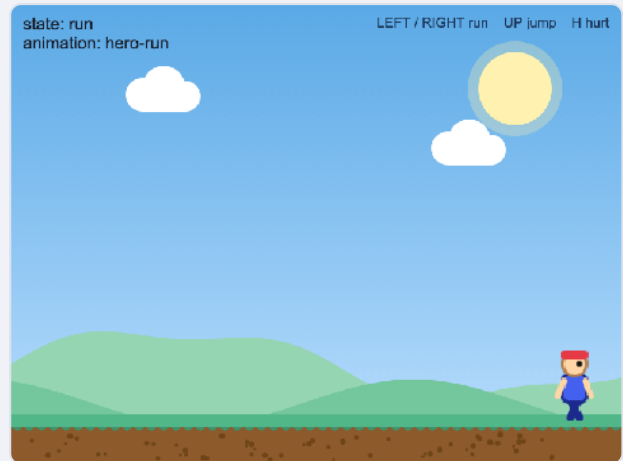
  this.once(Phaser.Animations.Events.ANIMATION_COMPLETE_KEY + HERO_HURT, () => {
    this.clearTint();
    this.changeState(this.isOnGround() ? "idle" : "fall");
  });
}

protected override preUpdate(time: number, delta: number): void {
  super.preUpdate(time, delta); // leave out: frozen on one frame, NO error
}
```

12 / 20

Try it: Hero Animations

- LEFT / RIGHT, UP, and H
- watch the state at the top
- change the hurt `frameRate` - what else changes?
- delete `super.preUpdate` - then put it back



13 / 20

Tweens

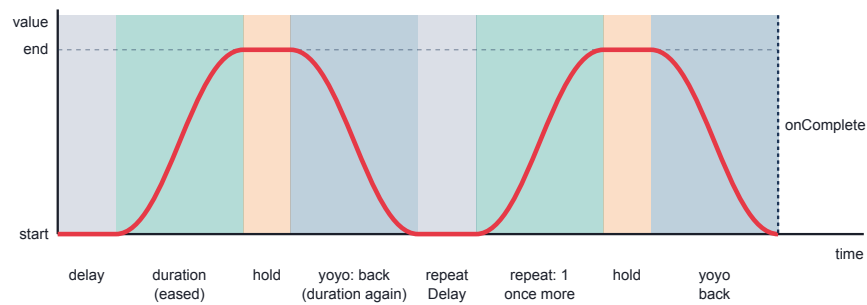
```
this.tweens.add({
  targets: this.crate,           // one object, or an array
  x: this.restX,                 // the values, and where they end up
  y: this.restY,
  duration: 800,                 // milliseconds
  ease: "Cubic.easeInOut",
});
```

- returns at once; Phaser does the changing, every frame
- relative values: `angle: "+=360"`
- `onComplete: () => popup.destroy()` - effects clean up after themselves

14 / 20

Yoyo, hold, repeat, delay

One tween, over time: { delay, duration, hold, yoyo: true, repeat: 1, repeatDelay }

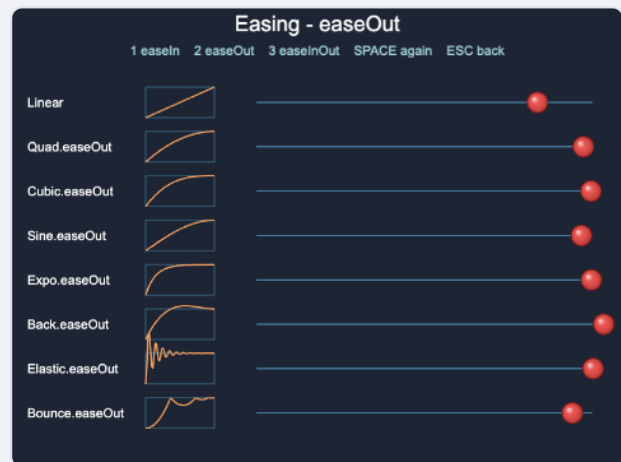


repeat: 2 = three times in all. repeat: -1 = for ever.

15 / 20

Easing

- easeIn - slow start (leaving)
- easeOut - slow finish (arriving)
- easeInOut - both (moving)
- Back, Elastic, Bounce - character
- misspelled ease = **linear**, silently



16 / 20

Chains, stagger, counters

```
this.tweens.chain({
  targets: this.crate,
  tweens: [
    { y: this.restY - 120, duration: 400, ease: "Quad.easeOut" },
    { angle: 360, duration: 500, ease: "Cubic.easeInOut" },
    { y: this.restY, duration: 700, ease: "Bounce.easeOut" },
    ...
  ],
});
```

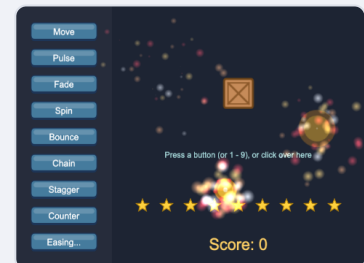
- **stagger**: one tween, many targets - `delay: this.tweens.stagger(60),`
- **counter**: a plain number - `this.tweens.addCounter({ from, to, onUpdate })`
- **no fighting**: `this.tweens.killTweensOf(this.crate);` before starting another

17 / 20

Particles

```
this.sparks = this.add.particles(0, 0, PARTICLE_KEY, {
  speed: { min: 80, max: 360 }, // random in a range
  ...
  scale: { start: 2.5, end: 0 }, // changes over its life
  ...
  tint: [0xffd166, 0xf4a261, 0xe63946, 0xffffffff], // one of these
  ...
  emitting: false, // made ONCE, silent...
});
this.sparks.explode(SPARKS, x, y); // ...until a burst

boom.once(Phaser.Animations.Events.ANIMATION_COMPLETE, () => {
  boom.destroy(); // the explosion sprite
  removes itself
});
```



18 / 20

Summary

- `this.anims.create + generateFrameNumbers` - once, for the game
- only a `Sprite` plays; `play(key, true); chain, playAfterRepeat, timeScale`
- animation **events**; `ANIMATION_COMPLETE` needs a finite `repeat`
- `super.preUpdate` in a `Sprite` subclass
- a **state machine**: one state, one place that changes it
- `this.tweens.add / chain / stagger / addCounter / killTweensOf`
- `this.add.particles(...), emitting: false, explode(n, x, y)`

19 / 20

Challenges

1. **Different speeds** - new frame rates; a `hero-sprint` on key 7
2. **Coin run** - spinning coins to collect, with a floating tween
3. **Title sequence** - one `tweens.chain`, restartable
4. **Sparkle trail** - particles that follow the pointer
5. **Speed you can see** - acceleration; stride matched with `timeScale`
6. **Stomp** - patrolling slimes: squash from above, hurt from the side

Next: **Chapter 8 - Collisions**

20 / 20